

4C11 Coursework 1

5630C

March 4, 2025

Abstract

Two physical problems are solved using neural networks. First, a load path problem for materials with and without path dependency. Second, determining whether a structure will fail depending on load distribution. Network design, hyperparameter tuning and performance is discussed. MLPs of varying depths are found to be the most powerful tool to solve these simple problems, but more complex datasets with more complex mappings may require more advanced architectures.

Problem 1

Design

The neural networks were designed to learn the constitutive model of three different materials (A, B and C). This investigation considered two possible material behaviours. Firstly a memoryless assumption that σ_t depends only on ϵ_t . Secondly a memory assumption, that σ_t depends on all previous $\epsilon_0, \dots, \epsilon_t$. In both the below investigations, the test/train split was 80/20%. The training target stresses are continuous values so mean squared error was used as the loss function. The optimiser was chosen to be AdamW, a version of Adam available in Pytorch that correctly implements weight decay.

Memoryless The mapping that the NN was to learn is

$$\psi_\theta =: \epsilon_t \mapsto \sigma_t \quad (1)$$

The neural network architecture used was a multi-layer perceptron (MLP), also known as a fully connected neural network. The strain state at time t is the input and the stress state at time t is the output. Thus for 3-dimensional materials A and B there are 6 input and output neurons (lower or upper diagonal of the stress/strain matrices), whereas for material C which is 1-dimensional there is 1 input and output neuron. There was one hidden layer with 100 neurons.

The hyperparameters tuned were: epochs; learning rate; weight decay; batch size.

Memory The mapping that the NN was to learn is

$$\psi_\theta =: \{\epsilon_i\}_{i=1}^t \mapsto \sigma_t \quad (2)$$

Two neural network architectures were used. Note, these tests were only carried out on material C so the stress and strain dimensions are 1 per time step.

First, an MLP with 50 input nodes and 50 output nodes, representing a full loading test time series. There is one hidden layer with 1000 neurons. Larger MLP variants with dropout were also tested.

Second, an LSTM with 1 input for each time step followed by a fully connected linear layer to combine the hidden states into 1 output.

The hyperparameters tuned were: epochs; learning rate; batch size. Additionally for the MLP: hidden layer number and size; dropout location and probability. Additionally for the LSTM: number of LSTM units; number of hidden states.

Results and Discussion

Memoryless An example of model predictions for one load test from the test set for the 3 materials is shown in fig. 1. The no memory approach worked well for materials A and B, suggesting that their behaviour is not path-dependent. Indeed, the very simple MLP with just 1 hidden layer and 100 nodes was sufficient to learn their behaviour.

Material C was different: the test loss was poor and did not decrease with larger network size, suggesting that the data is not sufficient to learn the model and path-dependency must be considered.

The optimal hyperparameters for the memoryless MLP were: epochs 30, learning rate 0.0005, weight decay 0, batch size 256. The validation set loss was $1.2e-5$ for material A, $2.5e-5$ for material B, and 0.42 for material C.

Memory Since materials A and B are well described by the no memory model, only material C was used to test the memory models. Examples of stress predictions of the MLP and LSTM models are shown in fig. 2(a,b). In terms of absolute loss, the MLP seems to outperform the LSTM. However, most of the LSTM loss comes from incorrect predictions of the first value of stress. fig. 3 shows that the first value of strain in the dataset is always zero. The LSTM model, which can only see previous inputs and not future inputs, must rely only on the first strain value to predict the first stress value, which is impossible. fig. 2(c) shows the training results when the output stresses are shifted one to the right i.e. $(\sigma_t \mapsto \sigma_{t+1})$, which are much improved and superior

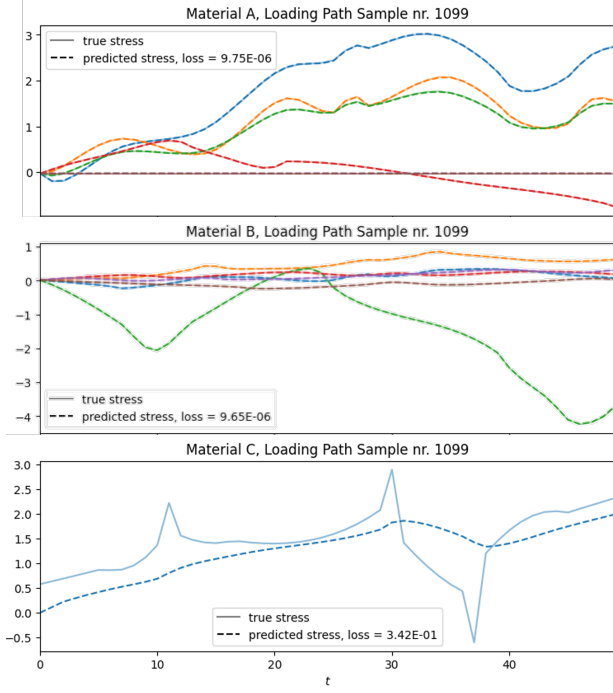


Figure 1: Comparison of model-predicted stress and true stress for 1 example from the test set with the memoryless model. Materials A and B have multiple stress/strain dimensions, each of these is its own trace. The memoryless model is not capable of describing the constitutive model of material C.

to the MLP (despite having fewer model parameters).

To demonstrate the path-dependent behaviour of material C, the LSTM model in fig. 2(c) is evaluated for a cyclic strain which rises linearly to 1 from 0, falls linearly to -1, and rises linearly to 0 again. This is tested for two different strain rates, as shown in fig. 4 ($n=40$ means the strain rate is double that of $n=80$). Material C seems to not only exhibit path dependency (i.e. memory), but also rate dependency.

The best test set mean square error loss was 0.030 for MLP and 0.0057 for LSTM. The optimal hyperparameters for the MLP were: batch size 16, epochs 50, learning rate 0.001. Larger MLP models and dropout did not improve. The optimal hyperparameters for the LSTM were: batch size 110, learning rate 0.005, epochs 200, weight decay 0, 5 LSTM units, 100 hidden states.

Problem 2

Design

The task was to predict whether a truss resembling the Eiffel tower, loaded by 20 selected loads (always in the same locations), would fail. Two examples from the dataset are given in ???. If b indicates whether the structure will fail or not, the mapping to

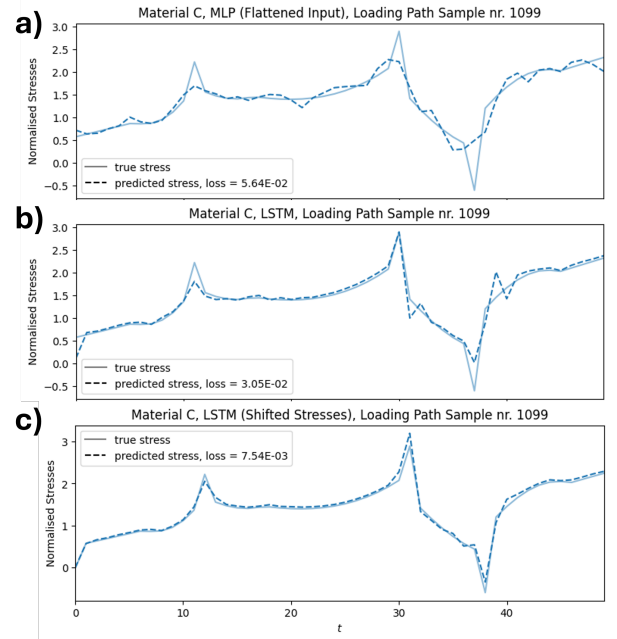


Figure 2: Comparison of model-predicted stress and true stress for 1 example from the test set with the memory models. Most of the LSTM error was caused by the first stress, which is impossible to predict using the first strain only.

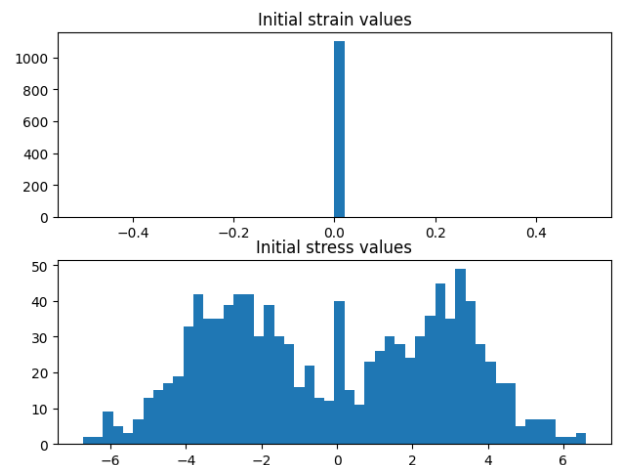


Figure 3: In the original dataset, the first strain value cannot by itself be used to predict the first stress value.

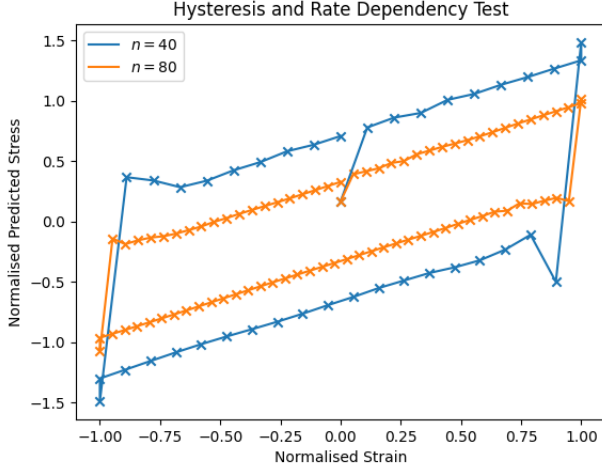


Figure 4: LSTM model in fig. 2(c) evaluated using a cyclic strain at two different strain rates to demonstrate hysteresis of material C. The two strain rates have different responses.

be learned was

$$\psi_{\theta} =: \{F_i\}_{i=1}^{20} \mapsto b \quad (3)$$

Two neural network architectures were tested: MLP and Deep Res-Net. In both the below investigations, the input dimension was 20 (the forces) and the output dimension was 1 (fail/not fail). The test/train split was 80/20%. The loss function was Binary Cross Entropy with Logits Loss, which is appropriate for binary classification. The optimiser was again AdamW.

Two MLP architectures were compared. The first had one hidden layer with 500 nodes. The second had 2 hidden layers with 500 nodes each, each with a 50% dropout rate.

One Deep Res-Net architecture is evaluated. Its architecture is shown in fig. 6. The skip connections (preventing vanishing gradients) and Batch Normalisation (stabilising training) allow for a very deep structure, which in theory allows for “richer” features to be extracted from complex data.

Data augmentation was attempted. Assuming the system behaves linearly, force magnitudes in surviving examples could be decreased to create new examples and force magnitudes in failing examples could be increased.

Results and Discussion

The attempt to augment the data was unsuccessful (very poor test accuracy and very high train accuracy), suggesting the failure behaviour of the system is nonlinear. The rest of the discussion pertains to results without the data augmentation. The loss and accuracy plots for the MLP and Deep Res-Net are shown in fig. 7.

MLP The MLP performs very well. Increasing the number of hidden layers to 2 and adding dropout in each layer significantly improved performance by allowing richer features to be learned and preventing overfitting respectively (see fig. 7).

The accuracy with dropout reached 99.5%, meaning only one example from the test set was predicted incorrectly. The loss reached 0.039. The optimal hyperparameters for the dropout MLP were: 2 hidden layers with 500 nodes each, 50% dropout probability in each layer, 500 epochs, learning rate 0.00005, batch size 100.

Deep Res-Net The Deep Res-Net, when dropout is included, matches the best MLP in terms of loss but has a somewhat smaller accuracy (0.985). The difference in accuracy could be caused by to randomness in selection of the test set, since the test set only has 200 examples. The advantage of the Res-Net is that it has fewer parameters (23,913) than the MLP (261,501), although the MLP was faster to train on my computer. With large scale data (such as high-resolution images) and more complex datasets, the required MLP size may become too large to train efficiently and a Res-Net would be more suitable.

Additionally, the convolutions involved in the Res-Net imply close relationship between impact of adjacent inputs which is slightly dubious here (but perfect for images or continuous materials, etc).

The optimal hyperparameters for the Res-Net were: learning rate 0.0001, batch size 100, epochs 125.

UNet Discussion I did not attempt to train a UNet for this problem. The premise of a UNet relies on “encoding” a high-dimensional input to a lower-dimensional latent space, in order to “decode” it again into an equivalent dimension, while aiding decoding with skip-connections. The high-dimensional output of a UNet means an additional structure, for example a linear layer, would be necessary at the output to output the binary classification needed.

I believe such an architecture is unlikely to outperform a ResNet or MLP in this scenario, and is better suited to other problems such as image transformation. Additionally, it would be difficult to verify or evaluate an improvement in performance over Res-Net and MLP with this dataset, as those networks have already almost reached the ceiling of possible accuracy.

Hyperparameter Tuning Strategy

This tuning strategy was similar for all tests, and also applies to question 2.

Initially, a minimal-size neural network architecture was chosen, and was tested using several combi-

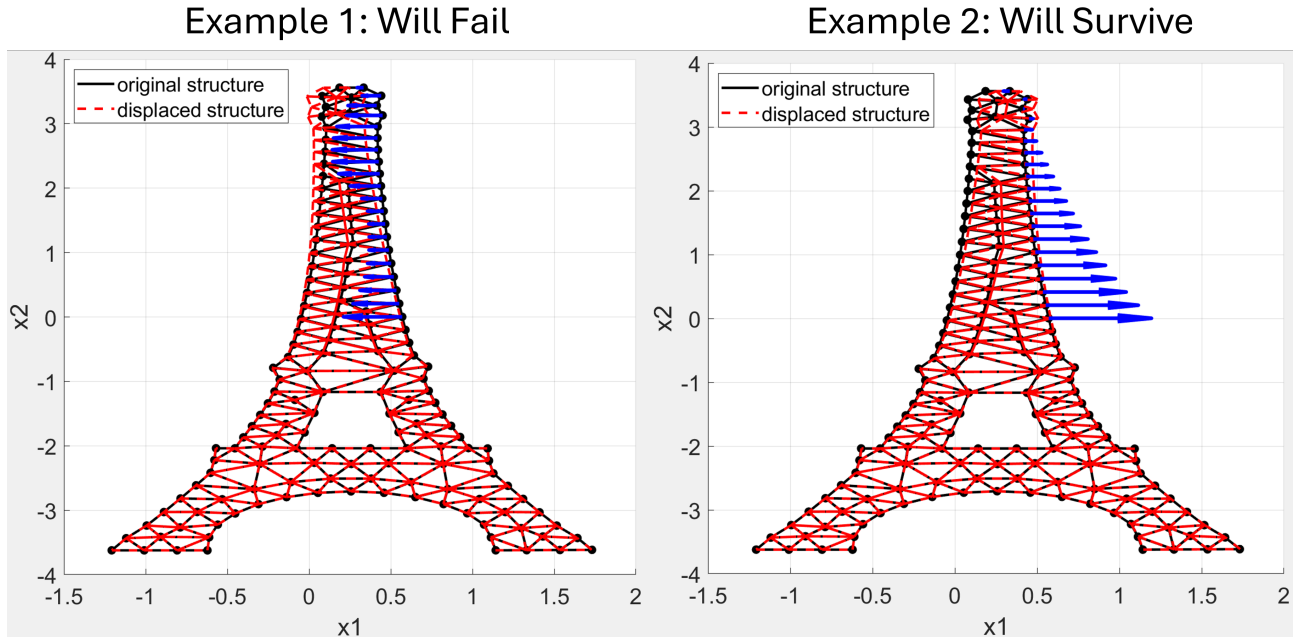


Figure 5: Two examples of loading from the dataset, indicating whether the structure will fail or not.

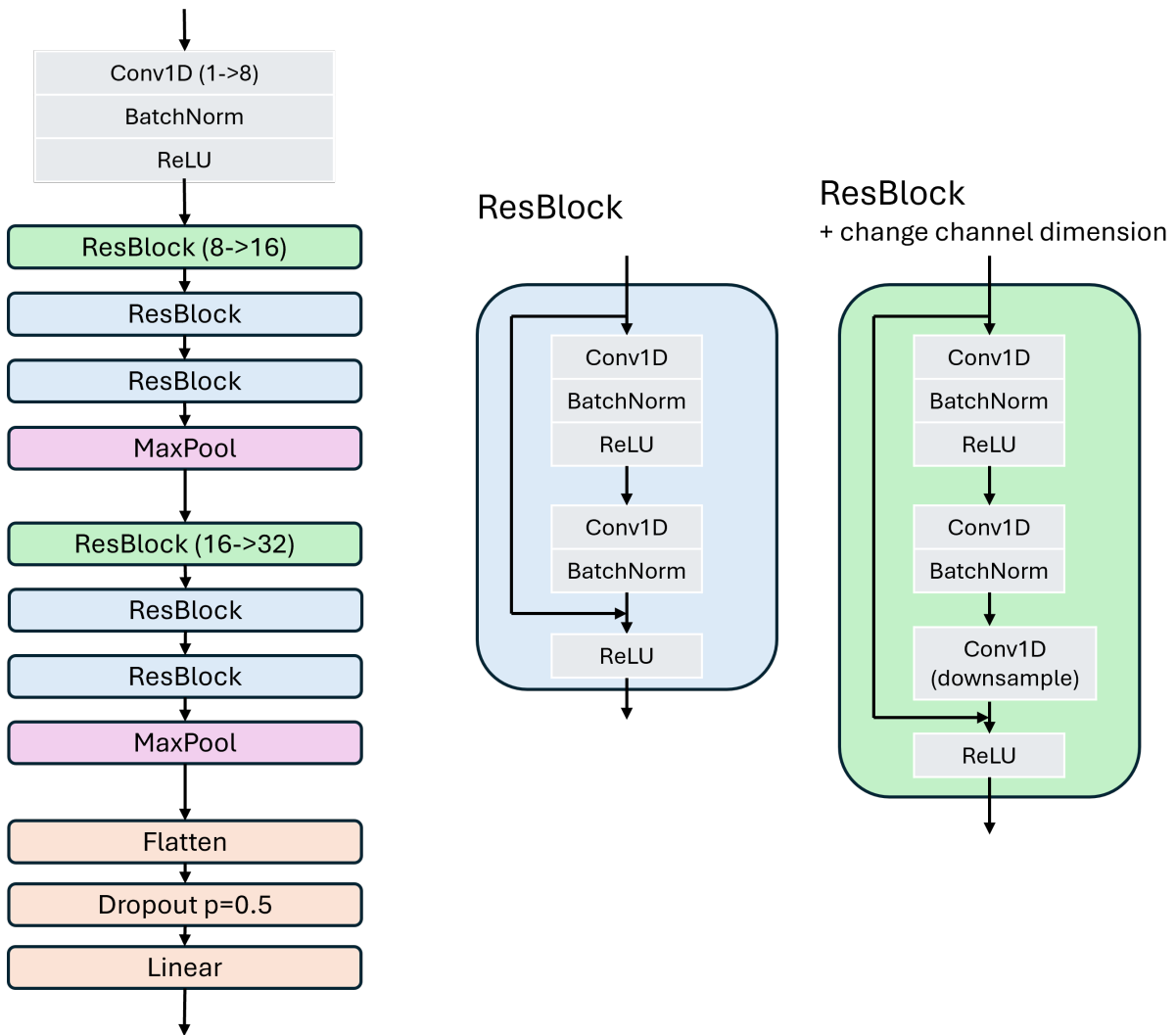


Figure 6: The Deep Res-Net architecture used. Right hand arrows in ResBlocks indicate up-scaling or down-scaling of the number of channels. Max Pooling involved pooling over 2 values with a step size of 2.

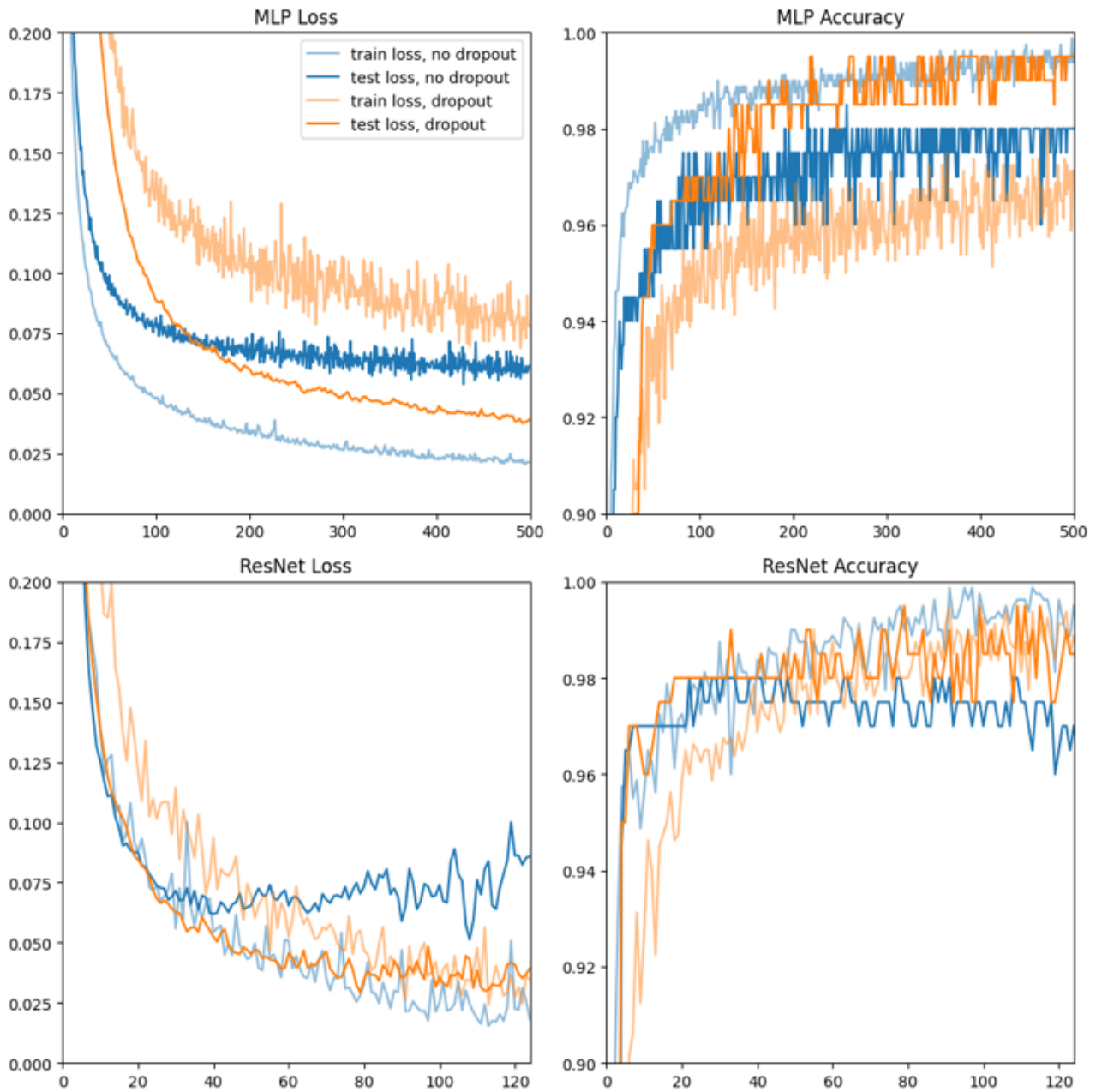


Figure 7: Loss (left) and accuracy (right) plots for 2-hidden-layer MLP (top) and ResNet (bottom) models, with and without dropout. Without dropout, the models reach a point where test loss increases and diverges from the train loss.

nations of training hyperparameters (usually learning rate, epochs, batch size) to find an optimal combination.

If overfitting was exhibited, which is common in deeper/larger networks, regularisation was attempted. In these tests the regularisation methods used were increasing weight decay and adding dropout to some fully connected layers. The optimal training hyperparameters were found again.

Once complete for the simple neural network architecture, the complexity of the architecture was increased and the tests repeated. This continued until the architecture no longer improved the test loss, or took too long to train. The complexity is increased like this for two reasons. Firstly, it is always better use the smallest (most efficient) possible model that can solve the problem equally well. Secondly, large models can be unwieldy and fiddly to train, so it is important to have a comparable baseline: what a simpler model is capable of.

There were a few general trends of note: Reducing batch size seemed to have a similar effect to increasing learning rate (allowing the model to achieve good results in fewer epochs) but increased training time and instability. Best results came from increasing the batch size and learning rate together, and running the experiment for a greater number of epochs (which is possible since increasing batch size increases training speed per epoch). Promising architectures and hyperparameter combinations benefitted significantly from a large number of epochs. Often, several hundred were needed before a plateau. Weight decay in the AdamW optimiser rarely had a noticeable effect. In large networks that without regularisation succumbed to overfitting, weight decay proved useless and dropout regularisation in fully connected layers was much more effective.